Disaster Management: Relief Distribution

Using Graph CRUD C

Data Structures — Final Project Report

Team Members

Member 1: S. Mohitha.

Member 2 : T. Geethika.

Abstract :

The Disaster Management System is a software application designed to manage disaster-related data efficiently using CRUD operations (Create, Read, Update, Delete). This project is implemented in C using dynamic memory allocation and structured programming. The system helps store and manage disaster records such as type, location, severity, and response actions. It ensures efficient handling of emergency data and improves decision-making during critical situations.

---

1. Introduction:

Disaster management is crucial for minimizing damage and ensuring quick recovery during emergencies. Managing large amounts of disaster-related data manually is inefficient. This project introduces a computerized system that allows users to store, update, and retrieve disaster information efficiently using data structures in C.

2.Objectives

• To implement CRUD operations using C

• To manage disaster-related data effectively

• To use dynamic memory allocation (malloc, free)

• To develop a menu-driven application

• To demonstrate practical use of data structures

---

3. Scope of the Project

The system can: • Store disaster details (earthquakes, floods, etc.)

• Update and delete records

• Display and search disaster data

Limitations:

• No database (memory-based storage only)

• No GUI (console-based system)

4. Literature Review

Existing systems use advanced databases and cloud platforms for disaster tracking. However, basic data structure-based systems are useful for learning and small-scale applications. Linked lists are commonly used for dynamic data storage due to flexibility.

---

5. Methodology

- Use linked list to store disaster records
- Each node contains:
  - Disaster ID
  - Name
  - Location
  - Severity Level
- Perform CRUD operations through menu options.

---

6. Technology Used

- Programming Language: C
- Concepts Used:
  - Structures (struct)
  - Pointers
  - Dynamic Memory Allocation
  - Functions

---

7. Architecture / System Design

- User interacts through menu
- Input is processed via functions
- Data stored in linked list
- Output displayed on console

---

8. Implementation Details

Structure Definition:

```
struct Disaster {
    int id;
    char name [50];
    char location [50];
    int severity;
    struct Disaster* next;
};
```

<u>CRUD Operations:</u>

- Create → Add new disaster record
- Read → Display all records
- Update → Modify existing record
- Delete → Remove record;

Menu Example:

1. Add Disaster.
2. Display Disasters.
3. Update Disaster.
4. Delete Disaster.
5. Exit.

---

9. Results / Output

Example Output:

Enter Disaster ID: 1

Enter Name: Flood.

Enter Location: Chennai

Enter Severity: 5

Disaster Added Successfully!
ID: 1 | Name: Flood | Location: Chennai | Severity: 5

---

10. Conclusion

The project successfully demonstrates how data structures can be used to manage real-world problems like disaster management. CRUD operations help maintain data efficiently, and the system can be expanded further for advanced use.

---

11. Future Work

- Add file handling or database storage
- Develop graphical user interface (GUI)
- Integrate real-time disaster data APIs
- Add alert and notification system

---

12. Footnotes / References

[1] A. Kumar, “Data Structures Using C,” McGraw-Hill, 2020.

[2] T. H. Cormen, “Introduction to Algorithms,” MIT Press, 2009.

[3] IEEE, “IEEE Citation Guidelines.” [Online]. Available: https://www.ieee.org